
Security Review Report

NM-0560 Lighter



NETHERMIND
SECURITY

(September 21, 2025)

Contents

1	Executive Summary	2
2	Audited Files	3
3	Summary of Issues	3
4	System Overview	4
4.1	ZkLighter Core Contract	4
4.2	AdditionalZkLighter Contract	4
4.3	Desert Mode Emergency System	4
4.4	Security Review Assumptions and Centralization Risks	4
5	Risk Rating Methodology	7
6	Issues	8
6.1	[Critical] Inconsistent hash padding permanently bricks emergency withdrawal mechanism	8
6.2	[Medium] Malicious deposits can Denial-of-Service critical address setters	10
6.3	[Info] Missing array length check in cancelOutstandingDepositsForDesertMode	11
6.4	[Info] Missing revert reason in upgradePreparationStarted()	11
6.5	[Info] Upgrade notice period is hardcoded to zero	11
6.6	[Best Practice] Critical role transfers should use a two-step process	12
6.7	[Best Practices] Critical functions lack event emissions	12
6.8	[Best Practices] _isContract check is unreliable with the Pectra upgrade	13
7	Documentation Evaluation	14
8	Test Suite Evaluation	15
8.1	Compilation Output	15
8.2	Tests Output	18
8.2.1	Foundry Tests Output	18
8.2.2	Hardhat Tests Output	19
9	About Nethermind	24

1 Executive Summary

This document presents the results of a security review conducted by [Nethermind Security](#) for [Lighter](#). The **Lighter** protocol facilitates a high-performance decentralized exchange (DEX) for perpetual futures trading, leveraging zero-knowledge proofs to achieve scalability while maintaining on-chain security guarantees.

The protocol operates as a Layer 2 rollup solution where users can deposit USDC collateral and engage in leveraged trading of perpetual contracts across various markets. All trading operations, including order placement, cancellation, and settlement, are processed off-chain by sequencers and then batch-committed to Ethereum mainnet through cryptographic proofs generated using zkSNARK technology.

Users interact with the protocol by depositing USDC into their trading accounts, which are managed through a sophisticated account system that tracks positions, collateral, and margin requirements across multiple markets. The protocol supports both limit and market orders, with real-time settlement and automatic liquidation mechanisms to maintain system solvency.

In addition to standard trading functionality, the protocol incorporates a **Desert Mode** emergency mechanism that activates when priority requests expire without execution. This mode allows users to perform trustless exits by providing cryptographic proofs of their account states, ensuring funds remain accessible even if the sequencer becomes unresponsive. The desert mode serves as a critical safety mechanism that preserves the protocol's decentralized guarantees and user fund security.

The audit comprises 1655 lines of Solidity code. The audit was performed using (a) manual analysis of the codebase, (b) automated analysis tools, and (c) creation of test cases.

Along this document, we report 8 points of attention, where one is classified as Critical, one is classified as Medium, three are classified as Informational and three are classified as Best Practices severity. The issues are summarized in Fig. 1.

This document is organized as follows. Section 2 presents the files in the scope. Section 3 summarizes the issues. Section 4 presents the system overview. Section 5 discusses the risk rating methodology. Section 6 details the issues. Section 7 discusses the documentation provided by the client for this audit. Section 8 presents the test suite evaluation and automated tools used. Section 9 concludes the document.

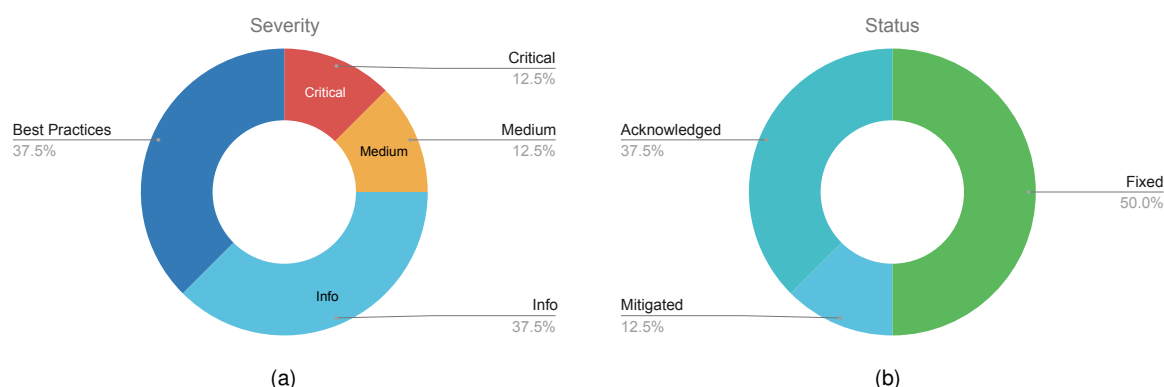


Fig. 1: Distribution of issues: Critical (1), High (0), Medium (1), Low (0), Undetermined (0), Informational (3), Best Practices (3). Distribution of status: Fixed (4), Acknowledged (3), Mitigated (1), Unresolved (0)

Summary of the Audit

Audit Type	Security Review
Initial Report	July 11, 2025
Final Report	September 21, 2025
Repository	Lighter
Initial Commit	8144649
Final Commit	0ddf2bf
Documentation	Whitepaper
Documentation Assessment	High
Test Suite Assessment	High

2 Audited Files

	Contract	LoC	Comments	Ratio	Blank	Total
1	DeployFactory.sol	77	11	14.3%	22	110
2	AdditionalZkLighter.sol	415	60	14.5%	64	539
3	Config.sol	38	40	105.3%	34	112
4	Governance.sol	60	26	43.3%	17	103
5	ZkLighter.sol	407	73	17.9%	59	539
6	Storage.sol	95	37	38.9%	28	160
7	UpgradeableMaster.sol	49	31	63.3%	18	98
8	proxy/Ownable.sol	28	15	53.6%	7	50
9	proxy/IUpgradeable.sol	4	6	150.0%	2	12
10	proxy/Proxy.sol	58	31	53.4%	13	102
11	proxy/UpgradeGatekeeper.sol	79	24	30.4%	20	123
12	proxy/IUpgradeEvents.sol	12	8	66.7%	6	26
13	lib/Bytes.sol	29	9	31.0%	6	44
14	lib/TxTypes.sol	177	18	10.2%	27	222
15	interfaces/IEvents.sol	21	20	95.2%	18	59
16	interfaces/IDesertVerifier.sol	4	7	175.0%	2	13
17	interfaces/IZkLighterDesertMode.sol	5	5	100.0%	3	13
18	interfaces/IGovernance.sol	16	20	125.0%	14	50
19	interfaces/IZkLighterVerifier.sol	4	7	175.0%	2	13
20	interfaces/IZkLighter.sol	77	106	137.7%	56	239
	Total	1655	554	33.5%	418	2627

3 Summary of Issues

	Finding	Severity	Update
1	Inconsistent hash padding permanently bricks emergency withdrawal mechanism	Critical	Fixed
2	Malicious deposits can Denial-of-Service critical address setters	Medium	Mitigated
3	Missing array length check in cancelOutstandingDepositsForDesertMode	Info	Fixed
4	Missing revert reason in upgradePreparationStarted()	Info	Fixed
5	Upgrade notice period is hardcoded to zero	Info	Acknowledged
6	Critical role transfers should use a two-step process	Best Practices	Acknowledged
7	Critical functions lack event emissions	Best Practices	Acknowledged
8	_isContract check is unreliable with the Pectra upgrade	Best Practices	Fixed

4 System Overview

The zkLighter protocol is a high-performance decentralized exchange (DEX) for perpetual futures trading, built as a zero-knowledge rollup solution on Ethereum. The protocol leverages zkSNARK technology to achieve scalability while maintaining on-chain security guarantees, enabling users to trade leveraged perpetual contracts with USDC as the primary collateral currency.

At the core, the protocol revolves around the ZkLighter contract, which implements the main rollup functionality including batch commitment, verification, and state management. The system processes trading operations off-chain through sequencers and periodically commits batched transactions to Ethereum mainnet via cryptographic proofs. To optimize gas costs and enable modular functionality, the protocol employs a dual-contract architecture where the AdditionalZkLighter contract handles user-facing operations such as deposits, withdrawals, priority request processing, and market management.

The protocol operates through a sophisticated account system that tracks user positions, collateral, and margin requirements across multiple perpetual markets. Users deposit USDC collateral into their accounts and can place limit or market orders, with real-time settlement and automatic liquidation mechanisms ensuring system solvency. The ZkLighterVerifier contract validates the cryptographic proofs generated by the sequencer, ensuring all state transitions are mathematically verified before being accepted on-chain.

A critical component of the protocol is the Desert Mode emergency mechanism, which activates when priority requests expire without execution. This mode allows users to perform trustless exits by providing cryptographic proofs of their account states through the DesertVerifier contract, ensuring funds remain accessible even if sequencers become unresponsive. The desert mode serves as a crucial decentralization guarantee, preventing sequencer censorship and maintaining user fund security.

The protocol's governance and upgrade mechanism is managed through the Governance contract and UpgradeGatekeeper system, which implements time-locked upgrades with notice periods. However, this introduces centralization concerns as the governance system has significant control over protocol parameters, market creation, and upgrade processes. The protocol owner can create new trading markets, adjust fees, and modify critical system parameters, representing potential centralization risks that users should be aware of.

4.1 ZkLighter Core Contract

The ZkLighter contract serves as the main rollup coordinator, implementing the core zero-knowledge proof verification and state management functionality.

- `commitBatch` function allows validators to commit new batches of transactions with cryptographic commitments to blob data.
- `verifyBatch` function validates zkSNARK proofs to ensure state transitions are mathematically correct before acceptance.
- `executeBatches` function processes verified batches to update on-chain state and execute withdrawal operations.
- Implements strict sequential processing to maintain rollup integrity and prevent state inconsistencies.

4.2 AdditionalZkLighter Contract

The AdditionalZkLighter contract handles user-facing operations and priority request management, serving as the primary interface for user interactions.

- Manages USDC deposits and withdrawal requests through priority queue mechanisms.
- Facilitates market creation and updates, with governance-controlled parameters and fees.
- Processes order placement, cancellation, and account management operations.
- Implements the desert mode cancellation mechanism for emergency fund recovery.

4.3 Desert Mode Emergency System

The Desert Mode provides a critical safety mechanism that activates when the sequencer fails to process priority requests within the expiration timeframe.

- `activateDesertMode` function can be called by anyone when priority requests expire, triggering emergency mode.
- `performDesert` allows users to prove their account states using cryptographic proofs and withdraw funds directly.
- `cancelOutstandingDepositsForDesertMode` enables cancellation of unprocessed deposits during emergency situations.
- Ensures protocol remains decentralized and censorship-resistant even during sequencer failures.

4.4 Security Review Assumptions and Centralization Risks

This security review was conducted under several key assumptions regarding the protocol's operational environment, trusted components, and privileged roles. These assumptions are critical to understanding the scope and limitations of the security analysis.

Privileged Roles Trust Assumptions: The review assumes that all privileged roles will operate honestly and will not act maliciously. The protocol incorporates several centralized roles with significant control:

- **Network Governor:** Holds the highest authority and can create/update market parameters (fees, margin requirements, interest rates), modify market status, change treasury and insurance fund operator addresses, manage validators, and transfer governance control
- **Validators:** Responsible for core rollup operations including committing batches, verifying proofs, executing on-chain operations, and reverting batches
- **Sequencer:** Controls transaction ordering, batch creation, and determines which transactions are included or excluded from batches. A malicious sequencer could censor transactions and manipulate ordering.
- **Security Council:** Can reduce upgrade notice periods from 3 weeks to zero for urgent fixes and expedite critical protocol upgrades
- **Upgrade Gatekeeper:** Manages protocol upgrades with authority to initiate, manage, execute, or cancel upgrades
- **Treasury and Insurance Fund Operators:** Control significant protocol funds, with the treasury collecting all protocol fees and the insurance fund operator managing liquidation proceeds

Malicious behavior by these privileged roles could result in market manipulation, transaction censorship, unauthorized upgrades, theft of protocol funds, or disruption of rollup operations.

Zero-Knowledge Circuit Integrity: The analysis assumes that the underlying zero-knowledge circuits used for batch verification and desert mode proofs are correctly implemented and free from vulnerabilities. Circuit bugs could enable invalid state transitions, unauthorized fund withdrawals, balance manipulation, or bypass of protocol constraints.

Desert Mode Recovery Completeness: The review assumes that the desert mode mechanism should allow users to recover all funds they are entitled to, including account balances, unrealized profits, pending transactions, and public pool shares. However, the review identified that certain fund types (such as public pool shares) cannot be recovered in desert mode due to the burnShares function being disabled when desert mode is active.

Upgrade Implementation Security: The review assumes that new contract implementations deployed through the upgrade mechanism are secure and do not introduce vulnerabilities. Malicious or buggy upgrade implementations could compromise the protocol.

Violations of these assumptions could lead to fund loss, protocol unavailability, or other security incidents beyond the scope of this smart contract security review. The centralized nature of key protocol functions creates potential single points of failure that users should consider when evaluating the protocol's risk profile.

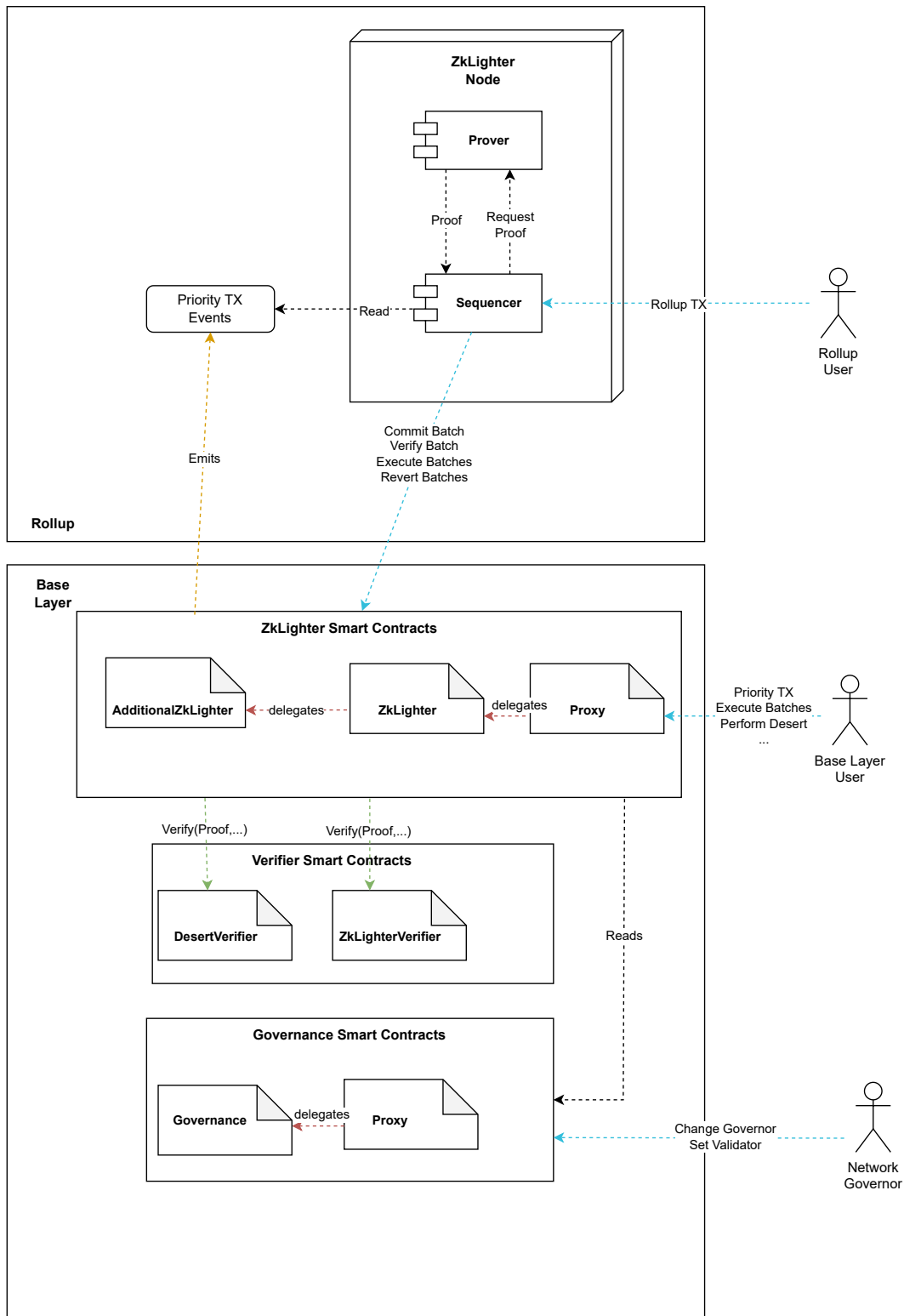


Fig. 2 The diagram shows the architecture and ZkLighter Interaction Diagram

5 Risk Rating Methodology

The risk rating methodology used by [Nethermind Security](#) follows the principles established by the [OWASP Foundation](#). The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

- a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;
- b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;
- c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

- a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;
- b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;
- c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Severity Risk		
Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Info/Best Practices	Low	Medium
	Undetermined	Undetermined	Undetermined	Undetermined
		Low	Medium	High
		Likelihood		

To address issues that do not fit a High/Medium/Low severity, [Nethermind Security](#) also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

- a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;
- b) **Best Practice** findings are used when some piece of code does not conform with smart contract development best practices;
- c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

6 Issues

6.1 [Critical] Inconsistent hash padding permanently bricks emergency withdrawal mechanism

File(s): `contracts/AdditionalZkLighter.sol`

Description: The `cancelOutstandingDepositsForDesertMode(...)` function is a critical emergency mechanism designed to allow users to reclaim their pending L1 deposits if the protocol enters "desert mode" (i.e., if L2 operations have stalled). The correct operation of this function relies on its ability to re-compute and validate a running hash of priority requests against the hash stored on-chain when the requests were first submitted.

When a priority request, such as a deposit, is made, the `addPriorityRequest(...)` function calculates a running hash (`prefixHash`) and stores it. This calculation is performed on the raw, un-padded bytes of the request data.

```

1 // contracts/AdditionalZkLighter.sol
2
3 function addPriorityRequest(TxTypes.PriorityPubDataType _pubdataType, bytes memory _priorityRequest, bytes memory
4   → _pubData) internal {
5     // ...
6     uint64 nextPriorityRequestId = executedPriorityRequestCount + openPriorityRequestCount;
7     bytes32 pubDataPrefix = bytes32(0);
8     if (nextPriorityRequestId > 0) {
9       pubDataPrefix = priorityRequests[nextPriorityRequestId - 1].prefixHash;
10    }
11    // ...
12    priorityRequests[nextPriorityRequestId] = PriorityRequest({
13      // @audit The `prefixHash` is computed using the raw, un-padded `_priorityRequest` bytes.
14      prefixHash: keccak256(abi.encodePacked(pubDataPrefix, _priorityRequest)),
15      expirationTimestamp: expirationTimestamp
16    });
17    // ...
18  }

```

However, during an emergency withdrawal via `cancelOutstandingDepositsForDesertMode(...)`, the logic attempts to verify the same request by first padding the `paddedPubData` to `MAX_PRIORITY_REQUEST_PUBDATA_SIZE` (54 bytes) and only then hashing it.

```

1 // contracts/AdditionalZkLighter.sol
2
3 function cancelOutstandingDepositsForDesertMode(uint64 _n, bytes[] memory _priorityPubData) external nonReentrant {
4   // ...
5   for (uint64 id = startIndex; id < startIndex + _n; ++id) {
6     // ...
7     bytes memory paddedPubData = new bytes(MAX_PRIORITY_REQUEST_PUBDATA_SIZE);
8     for (uint256 i = 0; i < _priorityPubData[currentPubDataIdx].length; ++i) {
9       paddedPubData[i] = _priorityPubData[currentPubDataIdx][i];
10    }
11
12    // @audit The hash is computed using the padded data.
13    pubDataPrefixHash = keccak256(abi.encodePacked(pubDataPrefixHash, paddedPubData));
14
15    // @audit This comparison will always fail for requests smaller than 54 bytes, like deposits.
16    if (pubDataPrefixHash != priorityRequests[id].prefixHash) {
17      revert AdditionalZkLighter_DepositPubdataHashMismatch();
18    }
19    // ...
20  }
21  // ...
22 }

```

This creates a fundamental inconsistency. A standard deposit request is only 35 bytes long. When `addPriorityRequest(...)` hashes the 35-byte array and `cancelOutstandingDepositsForDesertMode(...)` hashes a 54-byte padded array, the resulting hashes will never match. The comparison `pubDataPrefixHash != priorityRequests[id].prefixHash` will therefore always be true, causing the function to revert on the very first deposit it tries to cancel.

Since `cancelOutstandingDepositsForDesertMode(...)` is permanently broken, no user can ever cancel their pending deposits. The function's failure means `openPriorityRequestCount` cannot be decremented through this mechanism, and more importantly, the entire emergency withdrawal procedure for pending deposits is non-functional.

Recommendation(s): Consider aligning the hashing logic across both functions to ensure they operate on data with identical padding.

Status: Fixed

Update from the client: Fixed in commit [6cd7c8f](#).

6.2 [Medium] Malicious deposits can Denial-of-Service critical address setters

File(s): `contracts/ZkLighter.sol`, `contracts/AdditionalZkLighter.sol`

Description: The protocol uses the `setTreasury(...)` and `setInsuranceFundOperator(...)` functions, callable only by the governor, to configure critical addresses. These functions require that the address being set must not already have an associated account index within the system. This is enforced by the check `getAccountIndexFromAddress(...) != NIL_ACCOUNT_INDEX`.

```

1 // contracts/ZkLighter.sol
2
3 function setTreasury(address _newTreasury) external nonReentrant {
4     governance.requireGovernor(msg.sender);
5     if (_newTreasury == address(0)) {
6         revert ZkLighter_TreasuryCannotBeZero();
7     }
8     // @audit This check requires the new treasury to have no account index.
9     if (getAccountIndexFromAddress(_newTreasury) != NIL_ACCOUNT_INDEX) {
10         revert ZkLighter_TreasuryCannotBeInUse();
11     }
12     treasury = _newTreasury;
13     emit TreasuryUpdate(treasury);
14 }

```

However, the permissionless `deposit(...)` function in the `AdditionalZkLighter` contract allows any user to make a deposit to any arbitrary address (`_to`). If the recipient address does not already have an account index, the deposit flow registers the address, assigning it a new index.

```

1 // contracts/AdditionalZkLighter.sol
2
3 function deposit(uint64 _amount, address _to) external nonReentrant onlyActive {
4     // ...
5     // @audit Any address can be passed as `_to`.
6     if (_to == address(0)) {
7         revert AdditionalZkLighter_RecipientAddressInvalid();
8     }
9     // ...
10    // @audit This function will create an account index for `_to` if it doesn't exist.
11    registerDeposit(depositAmount, _to);
12 }

```

An attacker can exploit this design to mount a Denial-of-Service (DoS) attack on these setter functions. By monitoring the governor's activities, an attacker can identify a new address intended to be the next treasury or insurance fund operator. The attacker can then front-run the governor's transaction by calling `deposit(...)` with a nominal amount (e.g., 1 wei) and specifying the governor's new address as the recipient. This forces an account index to be created for the new address.

Consequently, when the legitimate `setTreasury(...)` or `setInsuranceFundOperator(...)` transaction is executed, its check will fail, causing it to revert. Since there is no way to "unregister" an account, this DoS will be permanent for the address.

Recommendation(s): Consider introducing a reasonably high minimum deposit amount to discourage these attacks.

Alternatively, consider implementing atomic deployment and configuration that creates the treasury/insurance fund contracts and sets them in a single transaction.

Status: Mitigated

Update from the client: Mitigated in commits [0ddf2bf](#) and [f4df0ed](#).

6.3 [Info] Missing array length check in cancelOutstandingDepositsForDesertMode

File(s): [contracts/AdditionalZkLighter.sol](#)

Description: The `cancelOutstandingDepositsForDesertMode(...)` function iterates `_n` times but lacks a check to ensure the length of the `_priorityPubData` input array is equal to `_n`. If an array with a length different from `_n` is provided, the call will revert unexpectedly due to an out-of-bounds access.

Recommendation(s): Consider adding a validation check for array lengths at the beginning of the function.

Status: Fixed

Update from the client: Fixed in commit [2545988](#).

6.4 [Info] Missing revert reason in upgradePreparationStarted()

File(s): [contracts/UpgradeableMaster.sol](#)

Description: The `require` statement in the `upgradePreparationStarted()` internal function lacks a descriptive error message. If this check fails, the transaction reverts without a clear reason, hindering debugging and monitoring efforts.

```
1 function upgradePreparationStarted() internal view {  
2     // @audit This require statement reverts without a specific error message.  
3     require(block.timestamp >= upgradeStartTimestamp + approvedUpgradeNoticePeriod);  
4 }
```

Recommendation(s): Consider adding an error message to the `require` statement to clarify the revert reason.

Status: Fixed

Update from the client: Fixed in commit [5a8745f](#).

6.5 [Info] Upgrade notice period is hardcoded to zero

File(s): [contracts/UpgradeableMaster.sol](#)

Description: An upgrade notice period is a critical security mechanism that provides users with a time window to review and react to upcoming protocol changes. However, the `getNoticePeriod()` function is currently hardcoded to return `0`.

```
1 function getNoticePeriod() internal pure returns (uint256) {  
2     return 0;  
3 }
```

This implementation completely nullifies the intended time-lock for upgrades.

Recommendation(s): Consider implementing the intended logic for the notice period.

Status: Acknowledged

6.6 [Best Practice] Critical role transfers should use a two-step process

File(s): [contracts/Governance.sol](#), [contracts/proxy/Ownable.sol](#)

Description: The protocol's critical administrative roles, such as the `networkGovernor` and `master`, are transferred using a direct, one-step process. The `changeGovernor(...)` function immediately assigns the new governor address upon being called.

```

1 // contracts/Governance.sol
2
3 function changeGovernor(address _newGovernor) external nonReentrant onlyGovernor {
4     if (_newGovernor == address(0)) {
5         revert ZkLighter_Governance_GovernorCannotBeZero();
6     }
7     // @audit The governor role is transferred in a single step.
8     if (networkGovernor != _newGovernor) {
9         networkGovernor = _newGovernor;
10        emit NewGovernor(_newGovernor);
11    }
12 }
```

Similarly, the `transferMastership(...)` function transfers the `master` role in a single transaction.

```

1 // contracts/proxy/Ownable.sol
2
3 function transferMastership(address _newMaster) external {
4     requireMaster(msg.sender);
5     require(_newMaster != address(0), "1d");
6     // @audit The master role is transferred in a single step.
7     setMaster(_newMaster);
8 }
```

This one-step pattern is dangerous. If the current owner makes a mistake and provides an incorrect address (e.g., due to a typo or copy-paste error), the administrative role will be transferred to an uncontrolled account.

Recommendation(s): Consider implementing a two-step process for all critical role transfers. OpenZeppelin's `Ownable2Step` contract is a good reference for this pattern.

Status: Acknowledged

6.7 [Best Practices] Critical functions lack event emissions

File(s): [contracts/AdditionalZkLighter.sol](#), [contracts/ZkLighter.sol](#), [contracts/proxy/Ownable.sol](#), [contracts/proxy/Proxy.sol](#)

Description: Emitting events for significant state changes is a crucial best practice. It allows off-chain services, monitoring tools, and user interfaces to reliably track important contract activities. Several functions across the codebase perform critical actions but lack event emissions.

The following functions modify critical state without event emissions:

- `performDesert(...)`: Increases a user's withdrawable balance and marks their account as having exited desert mode.
- `cancelOutstandingDepositsForDesertMode(...)`: Increases withdrawable balances for users whose deposits are canceled during desert mode.
- `transferMastership(...)`: Changes the master address.
- `upgrade(...)`: Changes the `additionalZkLighter` and `desertVerifier` contract addresses.
- `upgradeTarget(...)`: Changes the implementation target address for upgrades.

Recommendation(s): Consider adding event emissions to all functions that perform sensitive or critical state changes.

Status: Acknowledged

6.8 [Best Practices] `_isContract` check is unreliable with the Pectra upgrade

File(s): `contracts/Config.sol`

Description: The `_isContract()` check, which validates if `account.code.length > 0`, is no longer a reliable method for distinguishing smart contracts from EOAs. Since the Pectra network upgrade, EOAs can have associated code (per EIP-3074/EIP-7702), allowing them to pass this check.

```
1 // contracts/Config.sol
2
3 function _isContract(address account) internal view returns (bool) {
4     // @audit This check is unreliable as EOAs can now have code.
5     return account.code.length > 0;
6 }
```

The current use of this check in `initialize(...)` and `upgrade(...)` is sufficient to prevent input mistakes. However, due to the now misleading name of the function, future implementations of this function can cause problems as it might lead to thinking this function properly checks if the inputted address is a contract.

Recommendation(s): Consider renaming the function to avoid confusion.

Status: Fixed

Update from the client: Fixed in commit [09cb797](#).

7 Documentation Evaluation

Software documentation refers to the written or visual information that describes the functionality, architecture, design, and implementation of software. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Smart contracts can use various types of software documentation. Some of the most common types include:

- Technical whitepaper: A technical whitepaper is a comprehensive document describing the smart contract's design and technical details. It includes information about the purpose of the contract, its architecture, its components, and how they interact with each other;
- User manual: A user manual is a document that provides information about how to use the smart contract. It includes step-by-step instructions on how to perform various tasks and explains the different features and functionalities of the contract;
- Code documentation: Code documentation is a document that provides details about the code of the smart contract. It includes information about the functions, variables, and classes used in the code, as well as explanations of how they work;
- API documentation: API documentation is a document that provides information about the API (Application Programming Interface) of the smart contract. It includes details about the methods, parameters, and responses that can be used to interact with the contract;
- Testing documentation: Testing documentation is a document that provides information about how the smart contract was tested. It includes details about the test cases that were used, the results of the tests, and any issues that were identified during testing;
- Audit documentation: Audit documentation includes reports, notes, and other materials related to the security audit of the smart contract. This type of documentation is critical in ensuring that the smart contract is secure and free from vulnerabilities.

These types of documentation are essential for smart contract development and maintenance. They help ensure that the contract is properly designed, implemented, and tested, and they provide a reference for developers who need to modify or maintain the contract in the future.

Remarks about Lighter documentation

The Lighter team provided a technical whitepaper as the written documentation for the smart contracts for the scope of the audit and has provided a comprehensive walk-through of the project in the kick-off call with detailed explanation of the intended functionalities. Moreover, the team addressed all questions and concerns raised by the Nethermind Security team, providing valuable insights and a comprehensive understanding of the project's technical aspects.

8 Test Suite Evaluation

8.1 Compilation Output

```
> forge build
[] Compiling...
[] Compiling 62 files with Solc 0.8.25
[] Solc 0.8.25 finished in 22.17s
Compiler run successful with warnings:
Warning (5667): Unused function parameter. Remove or comment out the variable name to silence this warning.
--> contracts/ZkLighter.sol:117:20:
|
| 117 | function deposit(uint64 _amount, address _to) external {
|           ^^^^^^^^^^^^^^
|
Warning (5667): Unused function parameter. Remove or comment out the variable name to silence this warning.
--> contracts/ZkLighter.sol:117:36:
|
| 117 | function deposit(uint64 _amount, address _to) external {
|           ^^^^^^^^^^
|
Warning (5667): Unused function parameter. Remove or comment out the variable name to silence this warning.
--> contracts/ZkLighter.sol:122:25:
|
| 122 | function changePubKey(uint48 _accountIndex, uint8 _apiKeyIndex, bytes calldata _pubKey) external {
|           ^^^^^^^^^^^^^^^^^^^^^^^^^^
|
Warning (5667): Unused function parameter. Remove or comment out the variable name to silence this warning.
--> contracts/ZkLighter.sol:122:47:
|
| 122 | function changePubKey(uint48 _accountIndex, uint8 _apiKeyIndex, bytes calldata _pubKey) external {
|           ^^^^^^^^^^^^^^^^^^^^^^^^^^
|
Warning (5667): Unused function parameter. Remove or comment out the variable name to silence this warning.
--> contracts/ZkLighter.sol:122:67:
|
| 122 | function changePubKey(uint48 _accountIndex, uint8 _apiKeyIndex, bytes calldata _pubKey) external {
|           ^^^^^^^^^^^^^^^^^^^^^^^^^^
|
Warning (5667): Unused function parameter. Remove or comment out the variable name to silence this warning.
--> contracts/ZkLighter.sol:127:25:
|
| 127 | function createMarket(uint8 _size_decimals, uint8 _price_decimals, bytes32 _symbol, TxTypes.CreateMarket
|  |  ↳ calldata _params) external {
|           ^^^^^^^^^^^^^^^^^^^^^^
|
Warning (5667): Unused function parameter. Remove or comment out the variable name to silence this warning.
--> contracts/ZkLighter.sol:127:47:
|
| 127 | function createMarket(uint8 _size_decimals, uint8 _price_decimals, bytes32 _symbol, TxTypes.CreateMarket
|  |  ↳ calldata _params) external {
|           ^^^^^^^^^^^^^^^^^^^^^^
|
Warning (5667): Unused function parameter. Remove or comment out the variable name to silence this warning.
--> contracts/ZkLighter.sol:127:70:
|
| 127 | function createMarket(uint8 _size_decimals, uint8 _price_decimals, bytes32 _symbol, TxTypes.CreateMarket
|  |  ↳ calldata _params) external {
|           ^^^^^^^^^^^^^^^^^^^^^^
|
Warning (5667): Unused function parameter. Remove or comment out the variable name to silence this warning.
--> contracts/ZkLighter.sol:127:87:
|
| 127 | function createMarket(uint8 _size_decimals, uint8 _price_decimals, bytes32 _symbol, TxTypes.CreateMarket
|  |  ↳ calldata _params) external {
|           ^^^^^^^^^^^^^^^^^^^^^^
|
Warning (5667): Unused function parameter. Remove or comment out the variable name to silence this warning.
--> contracts/ZkLighter.sol:132:25:
|
| 132 | function updateMarket(TxTypes.UpdateMarket calldata _params) external {
|           ^^^^^^^^^^^^^^^^^^^^^^^^^^
|
```

```

Warning (5667): Unused function parameter. Remove or comment out the variable name to silence this warning.
--> contracts/ZkLighter.sol:137:28:
|
137 |     function cancelAllOrders(uint48 _accountIndex) public {
|                                     ^^^^^^^^^^^^^^^^^^^^^^

Warning (5667): Unused function parameter. Remove or comment out the variable name to silence this warning.
--> contracts/ZkLighter.sol:142:21:
|
142 |     function withdraw(uint48 _accountIndex, uint64 _usdcAmount) external {
|                                     ^^^^^^^^^^^^^^^^^^^^^^

Warning (5667): Unused function parameter. Remove or comment out the variable name to silence this warning.
--> contracts/ZkLighter.sol:142:43:
|
142 |     function withdraw(uint48 _accountIndex, uint64 _usdcAmount) external {
|                                     ^^^^^^^^^^^^^^^^^^^^^^

Warning (5667): Unused function parameter. Remove or comment out the variable name to silence this warning.
--> contracts/ZkLighter.sol:147:24:
|
147 |     function createOrder(uint48 _accountIndex, uint8 _marketIndex, uint48 _baseAmount, uint32 _price, uint8 _isAsk,
|                                     ^^^^^^^^^^^^^^^^^^^^^^
↳ uint8 _orderType) external {
|                                     ^^^^^^^^^^^^^^^^^^^^^^

Warning (5667): Unused function parameter. Remove or comment out the variable name to silence this warning.
--> contracts/ZkLighter.sol:147:46:
|
147 |     function createOrder(uint48 _accountIndex, uint8 _marketIndex, uint48 _baseAmount, uint32 _price, uint8 _isAsk,
|                                     ^^^^^^^^^^^^^^^^^^^^^^
↳ uint8 _orderType) external {
|                                     ^^^^^^^^^^^^^^^^^^^^^^

Warning (5667): Unused function parameter. Remove or comment out the variable name to silence this warning.
--> contracts/ZkLighter.sol:147:66:
|
147 |     function createOrder(uint48 _accountIndex, uint8 _marketIndex, uint48 _baseAmount, uint32 _price, uint8 _isAsk,
|                                     ^^^^^^^^^^^^^^^^^^^^^^
↳ uint8 _orderType) external {
|                                     ^^^^^^^^^^^^^^^^^^^^^^

Warning (5667): Unused function parameter. Remove or comment out the variable name to silence this warning.
--> contracts/ZkLighter.sol:147:86:
|
147 |     function createOrder(uint48 _accountIndex, uint8 _marketIndex, uint48 _baseAmount, uint32 _price, uint8 _isAsk,
|                                     ^^^^^^^^^^^^^^^^^^^^^^
↳ uint8 _orderType) external {
|                                     ^^^^^^^^^^^^^^^^^^^^^^

Warning (5667): Unused function parameter. Remove or comment out the variable name to silence this warning.
--> contracts/ZkLighter.sol:147:101:
|
147 |     function createOrder(uint48 _accountIndex, uint8 _marketIndex, uint48 _baseAmount, uint32 _price, uint8 _isAsk,
|                                     ^^^^^^^^^^^^^^^^^^^^^^
↳ uint8 _orderType) external {
|                                     ^^^^^^^^^^^^^^^^^^^^^^

Warning (5667): Unused function parameter. Remove or comment out the variable name to silence this warning.
--> contracts/ZkLighter.sol:147:115:
|
147 |     function createOrder(uint48 _accountIndex, uint8 _marketIndex, uint48 _baseAmount, uint32 _price, uint8 _isAsk,
|                                     ^^^^^^^^^^^^^^^^^^^^^^
↳ uint8 _orderType) external {
|                                     ^^^^^^^^^^^^^^^^^^^^^^

Warning (5667): Unused function parameter. Remove or comment out the variable name to silence this warning.
--> contracts/ZkLighter.sol:152:23:
|
152 |     function burnShares(uint48 _accountIndex, uint48 _publicPoolIndex, uint64 _shareAmount) external {
|                                     ^^^^^^^^^^^^^^^^^^^^^^

Warning (5667): Unused function parameter. Remove or comment out the variable name to silence this warning.
--> contracts/ZkLighter.sol:152:45:
|
152 |     function burnShares(uint48 _accountIndex, uint48 _publicPoolIndex, uint64 _shareAmount) external {
|                                     ^^^^^^^^^^^^^^^^^^^^^^

Warning (5667): Unused function parameter. Remove or comment out the variable name to silence this warning.
--> contracts/ZkLighter.sol:152:70:
|
152 |     function burnShares(uint48 _accountIndex, uint48 _publicPoolIndex, uint64 _shareAmount) external {
|                                     ^^^^^^^^^^^^^^^^^^^^^^

```

```

Warning (5667): Unused function parameter. Remove or comment out the variable name to silence this warning.
--> contracts/ZkLighter.sol:157:26:
|
157 | function revertBatches(StoredBatchInfo[] memory _batchesToRevert, StoredBatchInfo memory _remainingBatch)
-> external {
|
|
Warning (5667): Unused function parameter. Remove or comment out the variable name to silence this warning.
--> contracts/ZkLighter.sol:157:69:
|
157 | function revertBatches(StoredBatchInfo[] memory _batchesToRevert, StoredBatchInfo memory _remainingBatch)
-> external {
|
|
Warning (5667): Unused function parameter. Remove or comment out the variable name to silence this warning.
--> contracts/ZkLighter.sol:162:26:
|
162 | function performDesert(uint48 _accountIndex, uint48 _masterAccountIndex, uint128 _totalAccountValue, bytes
-> calldata proof) external {
|
|
Warning (5667): Unused function parameter. Remove or comment out the variable name to silence this warning.
--> contracts/ZkLighter.sol:162:48:
|
162 | function performDesert(uint48 _accountIndex, uint48 _masterAccountIndex, uint128 _totalAccountValue, bytes
-> calldata proof) external {
|
|
Warning (5667): Unused function parameter. Remove or comment out the variable name to silence this warning.
--> contracts/ZkLighter.sol:162:76:
|
162 | function performDesert(uint48 _accountIndex, uint48 _masterAccountIndex, uint128 _totalAccountValue, bytes
-> calldata proof) external {
|
|
Warning (5667): Unused function parameter. Remove or comment out the variable name to silence this warning.
--> contracts/ZkLighter.sol:162:104:
|
162 | function performDesert(uint48 _accountIndex, uint48 _masterAccountIndex, uint128 _totalAccountValue, bytes
-> calldata proof) external {
|
|
Warning (5667): Unused function parameter. Remove or comment out the variable name to silence this warning.
--> contracts/ZkLighter.sol:167:51:
|
167 | function cancelOutstandingDepositsForDesertMode(uint64 _n, bytes[] memory _depositsPubData) external {
|
|
Warning (5667): Unused function parameter. Remove or comment out the variable name to silence this warning.
--> contracts/ZkLighter.sol:167:62:
|
167 | function cancelOutstandingDepositsForDesertMode(uint64 _n, bytes[] memory _depositsPubData) external {
|
|
Warning (5667): Unused function parameter. Remove or comment out the variable name to silence this warning.
--> contracts/test/DesertVerifierTest.sol:8:19:
|
8 | function Verify(bytes calldata proof, uint256[] calldata public_inputs) external view returns (bool success) {
|
|
Warning (5667): Unused function parameter. Remove or comment out the variable name to silence this warning.
--> contracts/test/DesertVerifierTest.sol:8:41:
|
8 | function Verify(bytes calldata proof, uint256[] calldata public_inputs) external view returns (bool success) {
|
|
Warning (5667): Unused function parameter. Remove or comment out the variable name to silence this warning.
--> contracts/test/ZkLighterVerifierTest.sol:20:41:
|
20 | function Verify(bytes calldata proof, uint256[] calldata public_inputs) external view returns (bool success) {
|
|

```

```
Warning (2018): Function state mutability can be restricted to pure
--> contracts/test/DesertVerifierTest.sol:8:3:
|
|
8 |   function Verify(bytes calldata proof, uint256[] calldata public_inputs) external view returns (bool success) {
|   ^ (Relevant source part starts here and spans across multiple lines).

Warning (2018): Function state mutability can be restricted to view
--> contracts/test/UpgradeableMasterTest.sol:19:3:
|
|
19 |   function upgradePreparationStartedP() public {
|   ^ (Relevant source part starts here and spans across multiple lines).

Warning (2018): Function state mutability can be restricted to pure
--> contracts/test/ZkLighterVerifierTest.sol:20:3:
|
|
20 |   function Verify(bytes calldata proof, uint256[] calldata public_inputs) external view returns (bool success) {
|   ^ (Relevant source part starts here and spans across multiple lines).
```

8.2 Tests Output

8.2.1 Foundry Tests Output

```
> forge test

Ran 14 tests for test/ZkLighter.t.sol:ZkLighterTests
[PASS] test_commitBatch_fail_inactive_verifier() (gas: 85237)
[PASS] test_commitBatch_fail_invalid_batch_size() (gas: 86197)
[PASS] test_commitBatch_fail_invalid_end_block_number() (gas: 85548)
[PASS] test_commitBatch_fail_invalid_priority_prefix_hash() (gas: 147338)
[PASS] test_commitBatch_fail_invalid_pubdata_commitments() (gas: 67690)
[PASS] test_commitBatch_fail_invalid_pubdata_mode() (gas: 85509)
[PASS] test_commitBatch_fail_invalid_stored_batch() (gas: 96555)
[PASS] test_commitBatch_fail_invalid_timestamp() (gas: 86263)
[PASS] test_commitBatch_success() (gas: 181074)
[PASS] test_commitBatch_success_priority_and_onchain() (gas: 301080)
[PASS] test_verifyBatch_fail_invalid_batch() (gas: 62704)
[PASS] test_verifyBatch_fail_invalid_proof() (gas: 68187)
[PASS] test_verifyBatch_success() (gas: 214741)
[PASS] test_verifyBatch_success_priority_and_onchain() (gas: 441077)
Suite result: ok. 14 passed; 0 failed; 0 skipped; finished in 47.41ms (140.44ms CPU time)

Ran 1 test suite in 141.87ms (47.41ms CPU time): 14 tests passed, 0 failed, 0 skipped (14 total tests)
```

8.2.2 Hardhat Tests Output

```
> npx hardhat test
```

Solc version: 0.8.25	Optimizer enabled: true	Runs: 1000
Contract Name	Deployed size (KiB) (change)	Initcode size (KiB) (change)
AdditionalZkLighter	18.826 (0.000)	18.852 (0.000)
AdditionalZkLighterTest	18.859 (0.000)	18.885 (0.000)
Address	0.056 (0.000)	0.083 (0.000)
AddressUpgradeable	0.056 (0.000)	0.083 (0.000)
Bytes	0.056 (0.000)	0.083 (0.000)
Config	0.560 (0.000)	0.585 (0.000)
DeployFactory	0.056 (0.000)	27.388 (0.000)
DesertVerifier	6.985 (0.000)	7.011 (0.000)
DesertVerifierTest	0.232 (0.000)	0.257 (0.000)
ERC20	2.251 (0.000)	3.079 (0.000)
Faucet	1.313 (0.000)	1.755 (0.000)
Governance	2.183 (0.000)	2.370 (0.000)
GovernanceTest	2.310 (0.000)	2.497 (0.000)
KeccakTest	1.090 (0.000)	1.115 (0.000)
Ownable	0.415 (0.000)	0.606 (0.000)
OwnableERC20	2.523 (0.000)	3.408 (0.000)
Proxy	1.335 (0.000)	1.863 (0.000)
SafeCast	0.056 (0.000)	0.083 (0.000)
SafeERC20	0.056 (0.000)	0.083 (0.000)
Storage	1.663 (0.000)	1.688 (0.000)
TxTypes	0.056 (0.000)	0.083 (0.000)
UpgradeableMaster	0.691 (0.000)	0.965 (0.000)
UpgradeableMasterTest	1.256 (0.000)	1.538 (0.000)
UpgradeGatekeeper	4.020 (0.000)	4.384 (0.000)
ZkLighter	15.549 (0.000)	15.765 (0.000)
ZkLighterTest	16.656 (0.000)	16.889 (0.000)
ZkLighterVerifier	6.985 (0.000)	7.011 (0.000)
ZkLighterVerifierTest	0.346 (0.000)	0.371 (0.000)

Solc version: 0.8.25	Optimizer enabled: true	Runs: 1000
Contract Name	Deployed size (KiB) (change)	Initcode size (KiB) (change)
AdditionalZkLighter	18.826 (0.000)	18.852 (0.000)
AdditionalZkLighterTest	18.859 (0.000)	18.885 (0.000)
Address	0.056 (0.000)	0.083 (0.000)
AddressUpgradeable	0.056 (0.000)	0.083 (0.000)
Bytes	0.056 (0.000)	0.083 (0.000)
Config	0.560 (0.000)	0.585 (0.000)
DeployFactory	0.056 (0.000)	27.388 (0.000)
DesertVerifier	6.985 (0.000)	7.011 (0.000)
DesertVerifierTest	0.232 (0.000)	0.257 (0.000)
ERC20	2.251 (0.000)	3.079 (0.000)
Faucet	1.313 (0.000)	1.755 (0.000)
Governance	2.183 (0.000)	2.370 (0.000)
GovernanceTest	2.310 (0.000)	2.497 (0.000)
KeccakTest	1.090 (0.000)	1.115 (0.000)
Ownable	0.415 (0.000)	0.606 (0.000)
OwnableERC20	2.523 (0.000)	3.408 (0.000)
Proxy	1.335 (0.000)	1.863 (0.000)
SafeCast	0.056 (0.000)	0.083 (0.000)
SafeERC20	0.056 (0.000)	0.083 (0.000)
Storage	1.663 (0.000)	1.688 (0.000)
TxTypes	0.056 (0.000)	0.083 (0.000)
UpgradeableMaster	0.691 (0.000)	0.965 (0.000)
UpgradeableMasterTest	1.256 (0.000)	1.538 (0.000)
UpgradeGatekeeper	4.020 (0.000)	4.384 (0.000)
ZkLighter	15.549 (0.000)	15.765 (0.000)
ZkLighterTest	16.656 (0.000)	16.889 (0.000)
ZkLighterVerifier	6.985 (0.000)	7.011 (0.000)
ZkLighterVerifierTest	0.346 (0.000)	0.371 (0.000)

```

ZkLighter Tests
Deposit
  deposit USDC
    should reverted
    should success
    should register
    to treasury should success
    to insurance fund should success
Withdraw
  withdraw USDC
    should reverted
    should success
CancelAllOrders
  should reverted
  should success
CreateOrder
  should reverted
  should success
BurnShares
  should reverted
  should success
CreateMarket
  should create orderbook and emit `CreateMarket` event
  should fail to create orderbook if market index is invalid
  should fail to create orderbook if quoteMultiplier is invalid
  should fail to create orderbook if fees are invalid
  should fail to create orderbook if margin requirements are invalid
  should fail to create orderbook if interestRate is invalid
  should fail to create orderbook if min amounts are invalid
UpdateMarket
  should update orderbook and emit `UpdateMarket` event
  should fail to update orderbook if market index is invalid
  should fail to update orderbook if status is invalid
  should fail to update orderbook if fees are invalid
  should fail to update orderbook if margin requirements are invalid
  should fail to update orderbook if interestRate is invalid
  should fail to update orderbook if min amounts are invalid

Proxy
  Proxy contract should store target address
  Proxy contract should upgrade new target
    upgrade new `Governance` target
    upgradeTarget event
    upgrade new `ZkLighterVerifier` target
  Proxy contract should delegate function
    delegate `changeGovernor` function
    delegate `Verify` function
    delegate `deposit` function
    delegate `revertBatches` function
  Proxy contract should intercept upgrade function
    intercept `Governance` upgrade
    intercept `ZkLighterVerifier` upgrade
    intercept `ZkLighter` upgrade
  Destruct logic contract with `upgrade` method
    deploy ZkLighter
    - upgrade zkLighter
    invoking `upgrade` bypassing proxy should be intercepted

40 passing (1s)
1 pending

```

Solidity and Network Configuration					
Solidity: 0.8.25	Optim: true	Runs: 1000	viaIR: true	Block: 30,000,000 gas	
Methods					
Contracts / Methods	Min	Max	Avg	# calls	usd (avg)
Governance					
setValidator	-	-	55,141	2	-
GovernanceTest					
changeGovernor	-	-	35,786	1	-
setValidator	-	-	55,296	13	-
OwnableERC20					
approve	-	-	46,019	14	-
mint	53,196	70,308	55,649	14	-
Proxy					
upgradeTarget	32,732	33,016	32,886	4	-
ZkLighterTest					
burnShares	-	-	105,836	4	-
cancelAllOrders	-	-	100,492	4	-
createMarket	130,415	145,217	135,349	9	-
createOrder	-	-	107,254	4	-
deposit	145,016	204,841	196,432	52	-
revertBatches	-	-	110,895	1	-
setInsuranceFundOperator	-	-	53,835	1	-
setTreasury	-	-	71,397	1	-
updateMarket	-	-	140,986	3	-
withdraw	-	-	103,101	4	-
Deployments				% of limit	
AdditionalZkLighter	-	-	4,208,030	14 %	-
AdditionalZkLighterTest	-	-	4,215,313	14.1 %	-
DesertVerifierTest	-	-	104,883	0.3 %	-
Governance	-	-	560,710	1.9 %	-
GovernanceTest	-	-	588,823	2 %	-
OwnableERC20	-	-	692,844	2.3 %	-
Proxy	403,746	619,657	508,546	1.7 %	-
ZkLighterTest	-	-	3,750,301	12.5 %	-
ZkLighterVerifierTest	-	-	129,721	0.4 %	-
Key					

	☐ Execution gas for this method does not include intrinsic gas overhead
	Cost was non-zero but below the precision setting for the currency display (see options)
	Toolchain: hardhat

9 About Nethermind

Nethermind is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

Blockchain Security: At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

Blockchain Core Development: Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

DevOps and Infrastructure Management: Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

Cryptography Research: At Nethermind, our Cryptography Research team is dedicated to continuous internal research while fostering close collaboration with external partners. The team has expertise across a wide range of domains, including cryptography protocols, consensus design, decentralized identity, verifiable credentials, Sybil resistance, oracles, and credentials, distributed validator technology (DVT), and Zero-knowledge proofs. This diverse skill set, combined with strong collaboration between our engineering teams, enables us to deliver cutting-edge solutions to our partners and clients.

Smart Contract Development & DeFi Research: Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

Our suite of L2 tooling: Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;
- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

Learn more about us at nethermind.io.

General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

Disclaimer

This report is based on the scope of materials and documentation provided by you to [Nethermind](#) in order that [Nethermind](#) could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. [Nethermind](#) has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, [Nethermind](#) disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. [Nethermind](#) does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and [Nethermind](#) will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.